

Probability Theory and Optimization Concepts Reminder

Salvador Ruiz Correa

November 19th, 2024

Solve the following problem.

Box 1: Application Example (Was it the burglar?)

Mary lives in San Francisco. One afternoon, she is driving back home and receives a phone call from her neighbor Jane. She told her that her house alarm was set off (A). While driving, she also heard on the radio (R) that a small earthquake (E) hit the city. Small earthquakes sometimes activate the alarm, and perhaps this is the reason why the alarm was sounding.

- What is the probability that a burglar (B) broke into the house?
- What is the probability that a burglar broke into the house given that the alarm was set off?
- What is the probability that a burglar broke into the house given that the alarm was set off and a small earthquake hit the city?

To solve this problem, we make several considerations.

1. Define the random variables $R(R) = 1, R(R^c) = 0, A(A) = 1, A(A^c) = 0, E(E) = 1, E(E^c) = 0$, and $B(B) = 1, B(B^c) = 0$.
2. Assume that the joint distribution of random variables R, A, E , and B factorizes as

$$p(R, A, E, B) = p(E = e)p(B = b)p(A = a | B = b, E = e)p(R = r | E = e)$$

our using our simplified notation

$$p(r, a, e, b) = p(e)p(b)p(a | b, e)p(r | e).$$

The factors of the probability distribution correspond to the following tables.

$b \quad p(b)$		
B ^c	0	$b_1 = .9$
B	1	$b_2 = .1$

$e \quad p(e)$		
E ^c	0	$e_1 = 0.95$
E	1	e_2

$r \quad p(r)$		
R ^c	0	$c_1 = 0.05$
R	1	c_2

$b \quad e \quad p(b, e)$				
B ^c	E ^c	0	0	$b_1 e_1$
B ^c	E	0	1	$b_1 e_2$
B	E ^c	1	0	$b_2 e_1$
B	E	1	1	$b_2 e_2$

$r \quad e \quad p(r e)$				
R ^c	E ^c	0	0	$f_1 = 0.99$
R ^c	E	0	1	$f_2 = 0.01$
R	E ^c	1	0	f_3
R	E	1	1	f_4

$a \quad b \quad e \quad p(a b, e)$						
A ^c	B ^c	E ^c	0	0	0	q_1
A ^c	B ^c	E	0	0	1	q_2
A ^c	B	E ^c	0	1	0	q_3
A ^c	B	E	0	1	1	q_4
A	B ^c	E ^c	1	0	0	q_5
A	B ^c	E	1	0	1	q_6
A	B	E ^c	1	1	0	q_7
A	B	E	1	1	1	q_8

$$\begin{aligned}
q_1 + q_5 &= 1, \quad q_2 + q_6 = 1, \\
q_3 + q_5 &= 1, \quad q_4 + q_8 = 1, \\
b_1 + b_2 &= 1, \quad e_1 + e_2 = 1, \\
c_1 + c_2 &= 1, \\
f_1 + f_3 &= 1, \\
f_2 + f_4 &= 1.
\end{aligned}$$

The table representing the joint probability function is

$r \quad a \quad b \quad e \quad p(r, a, e, b)$								
R ^c	A ^c	B ^c	E ^c	0	0	0	0	$p_1 = b_1 e_1 f_1 p_1$
R ^c	A ^c	B ^c	E	0	0	0	1	$p_2 = b_1 e_2 f_1 p_2$
R ^c	A ^c	B	E ^c	0	0	1	0	$p_3 = b_2 e_1 f_1 p_3$
R ^c	A ^c	B	E	0	0	1	1	$p_4 = b_2 e_2 f_1 p_4$
R ^c	A	B ^c	E ^c	0	1	0	0	$p_5 = b_1 e_1 f_2 p_5$
R ^c	A	B ^c	E	0	1	0	1	$p_6 = b_1 e_2 f_2 p_6$
R ^c	A	B	E ^c	0	1	1	0	$p_7 = b_2 e_1 f_2 p_7$
R ^c	A	B	E	0	1	1	1	$p_8 = b_2 e_2 f_2 p_8$
R	A ^c	B ^c	E ^c	1	0	0	0	$p_9 = b_1 e_1 f_3 p_9$
R	A ^c	B ^c	E	1	0	0	1	$p_{10} = b_1 e_2 f_3 p_{10}$
R	A ^c	B	E ^c	1	0	1	0	$p_{11} = b_2 e_1 f_3 p_{11}$
R	A ^c	B	E	1	0	1	1	$p_{12} = b_2 e_2 f_3 p_{12}$
R	A	B ^c	E ^c	1	1	0	0	$p_{13} = b_1 e_1 f_4 p_{13}$
R	A	B ^c	E	1	1	0	1	$p_{14} = b_1 e_2 f_4 p_{14}$
R	A	B	E ^c	1	1	1	0	$p_{15} = b_2 e_1 f_4 p_{15}$
R	A	B	E	1	1	1	1	$p_{16} = b_2 e_2 f_4 p_{16}$

$$\begin{aligned}
q_1 + q_5 &= 1, \\
q_2 + q_6 &= 1, \\
q_3 + q_5 &= 1, \\
q_4 + q_8 &= 1, \\
b_1 + b_2 &= 1, \\
e_1 + e_2 &= 1, \\
c_1 + c_2 &= 1, \\
f_1 + f_3 &= 1, \\
f_2 + f_4 &= 1.
\end{aligned}$$

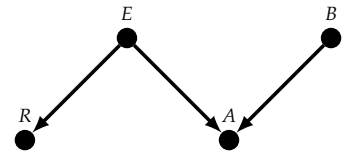


Figure 1: Directed Acyclic Graph encoding the probability distribution $p(r, a, e, b) = p(e)p(b)p(a | b, e)p(r | e)$. Dark circles represent random variables R , A , E , and B .

- The *graphical model* (a Directed Acyclic Graph or DAG) encoding

the probability distribution factorization is shown in Figure 21.

- Assume that $q_1 = 0.98$, $q_2 = 0.9$, $q_3 = 0.1$, $q_4 = 0.01$.
- What is the probability that a burglar broke into the house?

$$\begin{aligned} P(B = 1) &= P(B) = \\ &= p_3 + p_4 + p_7 + p_8 + p_{11} + p_{12} + p_{15} + p_{16}, \end{aligned}$$

$$P(B = 1) = \sum_a \sum_r \sum_e p(r, a, B = 1, e) = 0.1.$$

- What is the probability that a burglar broke into the house given that the alarm was set off?

$$\begin{aligned} P(B = 1 \mid A = 1) &= \frac{P(B = 1, A = 1)}{P(A = 1)} \\ &= \frac{p_7 + p_8 + p_{15} + p_{16}}{p_5 + p_6 + p_7 + p_8 + p_{13} + p_{14} + p_{15} + p_{16}}, \end{aligned}$$

$$P(B = 1 \mid A = 1) = \frac{\sum_r \sum_e p(r, A = 1, B = 1, e)}{\sum_r \sum_b \sum_e p(r, A = 1, b, e)} = 0.81.$$

- What is the probability that a burglar broke into the house given that the alarm was set off and a small earthquake hit the city?

$$\begin{aligned} P(B = 1 \mid A = 1, E = 1) &= \frac{P(B = 1, A = 1, E = 1)}{P(A = 1, E = 1)} \\ &= \frac{p_8 + p_{16}}{p_6 + p_8 + p_{14} + p_{16}}, \end{aligned}$$

$$\begin{aligned} P(B = 1 \mid A = 1, E = 1) &= \frac{\sum_r p(r, A = 1, B = 1, E = 1)}{\sum_r \sum_b p(r, A = 1, b, E = 1)} \\ &= 0.51. \end{aligned}$$

Box 2: Solution Summary

- What is the probability that a burglar broke into the house?

$$P(B = 1) = 0.1.$$

- What is the probability that a burglar broke into the house given that the alarm was set off?

$$P(B = 1 \mid A = 1) = 0.81.$$

- What is the probability that a burglar broke into the house given that the alarm was set off and a small earthquake hit the city?

$$P(B = 1 \mid A = 1, E = 1) = 0.51.$$

- The *a priori* probability that the burglar broke into the house is 0.1. The probability that a burglar broke into the house given that the alarm was set off is much higher (0.81). However, knowing that a small earthquake hit the city *explains away* the observation that the alarm was set off, diminishing the probability that a burglar broke (0.51).

- Note that the factorization features of the probability distribution and the sum-product distributive property help us reduce the computational complexity.

$$\begin{aligned}
 p(b \mid a) &= \frac{p(a, b)}{p(a)} = \frac{\sum_r \sum_e p(r, a, e, b)}{\sum_b \sum_r \sum_e p(r, a, e, b)} \\
 &= \frac{\sum_r \sum_e p(e) p(b) p(a \mid e, b) p(r \mid e)}{\sum_b \sum_r \sum_e p(e) p(b) p(a \mid e, b) p(r \mid e)} \\
 &= \frac{p(b) \sum_e p(e) p(a \mid e, b) \sum_r p(r \mid e)}{\sum_b p(b) \sum_e p(e) p(a \mid e, b) \sum_r p(r \mid e)} \\
 &= \frac{p(b) \sum_e p(e) p(a \mid e, b) \phi_1(e)}{\sum_b p(b) \sum_e p(e) p(a \mid e, b) \phi_1(e)} \\
 &= \frac{p(b) \phi_2(a, b)}{\sum_b p(b) \phi_2(a, b)} \\
 &= \frac{p(b) \phi_2(a, b)}{\phi_3(a)}
 \end{aligned}$$

Drawing Samples from Random Variable

Drawing samples from a random variable refers to generating individual realizations or instances of that random variable according

to its probability distribution. In probability theory and statistics, a random variable is a variable whose possible values are outcomes of a random phenomenon. The probability distribution of a random variable describes the likelihood of different values it can take.

When you draw samples from a random variable, you are essentially simulating or generating data points that follow the probability distribution of that variable. This process is often used in various fields, such as statistics, machine learning, and simulations, to understand random phenomena's behavior or make predictions.

Let X be a random variable and $p(x) := p_X(x)$ be the corresponding probability distribution. If X is a discrete random variable, then it takes values in a countable set.

For example, if you have a random variable representing the outcome of a fair six-sided die roll, drawing samples from this random variable would involve simulating the die roll and obtaining values like 1, 2, 3, 4, 5, or 6 with equal probability, i.e. $\Omega = \{1, 2, 3, 4, 5, 6\}$.

Drawing samples is fundamental to Monte Carlo simulations, where random sampling is used to estimate numerical results and analyze complex systems that involve randomness. In statistical terms, the more samples you draw, the better your approximation of the random variable's true underlying distribution or properties.

Consider the data data-generating process for N independent coin flips, where p is the heads probability. For $N = 2$ the joint probability distribution $p(x_1, x_2)$ is computed as follows.

$$x_1 \sim \text{Bernoulli}(x_1; q)$$

$$x_2 \sim \text{Bernoulli}(x_2; q)$$

that is to say,

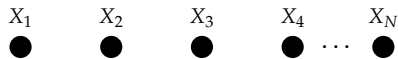
$$p(x_1) = \text{Bernoulli}(x_1; q),$$

$$p(x_2) = \text{Bernoulli}(x_2; q);$$

the joint probability distribution is

$$p(x_1, x_2) = p(x_1)p(x_2).$$

The following is the graphical joint probability distribution of the N i.i.d. variables.



The corresponding probability distribution is the following.

$$\begin{aligned}
 p(x_1, x_2, \dots, x_N) &= p(x_1)p(x_2) \cdots p(x_N) \\
 &= \prod_{n=1}^N p(x_n) \\
 &= \prod_{n=1}^N \text{Bernoulli}(x_n; q) \\
 &= \prod_{n=1}^N q^{x_n} (1 - q)^{1-x_n} \\
 &= q^{\sum_{n=1}^N x_n} (1 - q)^{\sum_{n=1}^N 1 - \sum_{n=1}^N x_n} \\
 &= q^r (1 - q)^{N-r},
 \end{aligned}$$

where $r = \sum_{n=1}^N x_n$ is the number of heads in N tosses. The log likelihood is

$$\text{LL}(q; \mathbf{x}) = \log p(x_1, x_2, \dots, x_N) = r \log q + (N - r) \log(1 - q).$$

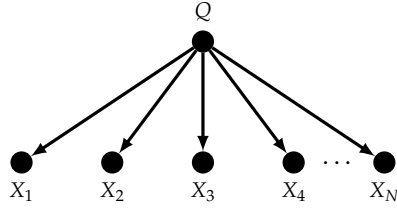
When q is unknown, the frequentist view proposes to estimate it by solving the following optimization problem.

$$\hat{q}_{\text{MLE}} = \arg \max_q \text{LL}(q; \mathbf{x})$$

$$\hat{q}_{\text{MLE}} = \frac{r}{N}.$$

This is the Maximum Likelihood Estimation (MLE) of *parameter* q .

In the Bayesian context, the head probability is considered a random variable Q . The corresponding graphical model is the following.



The corresponding probability distribution is the following.

$$\begin{aligned}
 p(x_1, x_2, \dots, x_N | q) &= p(x_1 | q)p(x_2 | q) \cdots p(x_N | q) \\
 &= \prod_{n=1}^N p(x_n | q) \\
 &= \prod_{n=1}^N \text{Bernoulli}(x_n | q) \\
 &= \prod_{n=1}^N q^{x_n} (1 - q)^{1-x_n} \\
 &= q^{\sum_{n=1}^N x_n} (1 - q)^{\sum_{n=1}^N 1 - \sum_{n=1}^N x_n} \\
 &= q^r (1 - q)^{N-r}.
 \end{aligned}$$

The posterior distribution of Q can be computed using the Bayes theorem.

$$\begin{aligned}
 p(q | x_1, x_2, \dots, x_N) &= \frac{p(x_1, x_2, \dots, x_N | q)p(q)}{p(x_1, x_2, \dots, x_N)} \\
 &= \frac{q^r (1 - q)^{N-r} p(q)}{p(x_1, x_2, \dots, x_N)} \\
 &\propto q^r (1 - q)^{N-r} p(q).
 \end{aligned}$$

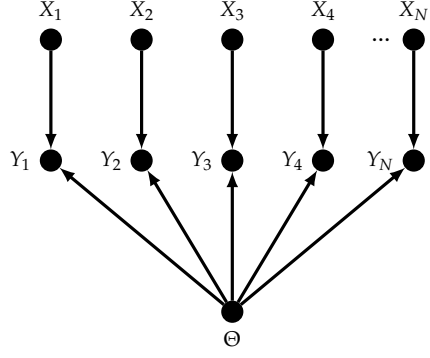
The Maximum a Posteriori (MAP) *point estimation* is defined as follows.

$$\begin{aligned}
 \hat{q}_{MAP} &= \arg \max_q \frac{p(x_1, x_2, \dots, x_N | q)p(q)}{p(x_1, x_2, \dots, x_N)} \\
 &= \arg \max_q q^r (1 - q)^{N-r} p(q).
 \end{aligned}$$

Machine learning classification problems require finding ML or MAP parameter estimates. Consider for example the following probability distribution relating feature vectors $\mathbf{x}_i$, labels y_i and model variables θ for an i.i.d. data set $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$.

$$\begin{aligned}
 p(\mathbf{y}, \mathbf{x}, \theta) &= p(y_1, y_2, \dots, y_N, x_1, x_2, \dots, x_N, \theta_1, \theta_2, \dots, \theta_N) \\
 &= p(y_1 | x_1, \theta)p(y_2 | x_2, \theta) \cdots p(y_N | x_N, \theta)p(x_1)p(x_2) \cdots p(x_N)p(\theta) \\
 &= p(\theta) \prod_{n=1}^N p(y_n | x_n, \theta)p(x_n)
 \end{aligned}$$

The corresponding graphical model is the following.



$$\begin{aligned}
 p(\mathbf{y} \mid \mathbf{x}, \theta) &= \frac{p(\mathbf{y}, \mathbf{x}, \theta)}{p(\mathbf{x}, \theta)} \\
 &= \frac{p(\theta) \prod_{n=1}^N p(y_n \mid x_n, \theta) p(x_n)}{\sum_{\mathbf{y}} p(\theta) \prod_{n=1}^N p(y_n \mid x_n, \theta) p(x_n)} \\
 &= \frac{p(\theta) \prod_{n=1}^N p(y_n \mid x_n, \theta) p(x_n)}{p(\theta) \prod_{n=1}^N p(x_n)} \\
 &= \prod_{n=1}^N p(y_n \mid x_n, \theta).
 \end{aligned}$$

The corresponding log-likelihood is

$$\begin{aligned}
 \text{LL}(\mathbf{y} \mid \mathbf{x}, \theta) &= \log p(\mathbf{y} \mid \mathbf{x}, \theta) \\
 &= \sum_{i=1}^N \log p(y_i \mid x_i, \theta).
 \end{aligned}$$

We use an alternative notation to denote the above probability as follows to clarify the definition of the *cross-entropy error function* described in the section that follows.

$$p(\mathbf{y} \mid \mathbf{x}, \theta) = p_{Y|X, \Theta}(\mathbf{y} \mid \mathbf{x}, \theta).$$

The likelihood is

$$\text{LL}(\mathbf{y} \mid \mathbf{x}, \theta) = \sum_{n=1}^N \log_2 p_{Y|X, \Theta}(y_n \mid x_n, \theta).$$

Cross-Entropy Error Function

Let $p_{Y|X,\theta}$, be a probability distribution approximating $p_{Z|X}$, where $Y = f(Z)$. Recall that

$$\begin{aligned} H(p_{Z|X}, p_{Y|X,\theta}) &= - \sum_z p_{Z|X}(z) \log_2 p_{Y|X,\theta}(z) \\ &= H_{Z|X}(p_{Z|X}) + D_{KL}(p_{Z|X} \parallel p_{Y|X,\theta}) \\ &= -E_{Z|X} \left[\log_2 p_{Y|X,\theta}(Y | X, \theta) \right] \end{aligned}$$

An optimal approximation of $p_{Z|X}$ w.r.t. Θ can be obtained by minimizing $D_{KL}(p_{Z|X} \parallel p_{Y|X,\theta})$, since $H_{Z|X}(Z)$ does not depend on θ . This is equivalent to minimizing the cross-entropy $H(p_{Z|X}, p_{Y|X,\theta})$.

The question is how to estimate $-E_{Z|X} \left[\log_2 p_{Y|X,\theta}(Y | X, \theta) \right]$. We draw a set of *i.i.d.* samples $X_1, X_2, \dots, X_N \sim p_X$, $Z_n \sim p_{Z|X_n}$, with $Y_n = f(Z_n)$ for $n = 1, \dots, N$, and compute the sample mean

$$\begin{aligned} -\frac{1}{N} \sum_{n=1}^N \log_2 p_{Y|X,\theta}(y_n | x_n, \theta) &= -\frac{1}{N} LL(\mathbf{y} | \mathbf{x}, \theta) \\ &\approx -E_{Z|X} \left[\log_2 p_{Y|X,\theta}(Y) \right] \\ &= H(p_{Z|X}, p_{Y|X,\theta}) \end{aligned}$$

which implies that

$$LL(\mathbf{y} | \mathbf{x}, \theta) \rightarrow -NH(p_{Z|X}, p_{Y|X,\theta})$$

as N tends to infinity. Therefore, maximizing the log-likelihood of the data w.r.t. θ is equivalent to minimizing the cross-entropy function w.r.t. Θ .

Let $\hat{\theta}$ be the estimated parameter after minimization. If $D_{KL}(p_{Z|X} \parallel p_{Y|X,\hat{\theta}})$ is close to zero, then

$$LL(\mathbf{y} | \mathbf{x}, \hat{\theta}) \approx -NH(p_{Z|X}, p_{Y|X,\hat{\theta}})$$

and therefore

$$\prod_{n=1}^N p_{Y|X,\hat{\theta}}(X_n) \approx 2^{-NH_{Z|X}(Z)}.$$

Choosing a Classification Function

We are given a training set $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$, where $y \in \{1, -1\}$, $y_n = f(\mathbf{x}_n)$, for $n = 1, \dots, N$. We want to use $\mathcal{D}$ to construct a classification function that can predict the correct label y_{new} given the input features $\mathbf{x}_{new}$.

The linear model for classifying data into two classes uses a hypothesis set of linear classifiers. The hypothesis space is defined as

$$\mathcal{H} = \{h(\mathbf{x}) : h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x}), \mathbf{w} \in \mathbb{R}^{d+1}\},$$

where d is the dimensionality of the input space. Notice that each element of $\mathcal{H}$ is specified by a $\mathbf{w}$ vector. We say that $\mathcal{H}$ is parametrized by $\mathbf{w}$ (i.e., $\mathcal{H} = \mathcal{H}_{\mathbf{w}}$).

The added coordinate $x_0 = 1$ corresponds to the bias weight w_0 . More specifically: $\mathbf{x} = (1, x_1, x_2, \dots, x_d)$ and $\mathbf{w} = (w_0, w_1, w_2, \dots, w_d)$. Recall that

$$h(\mathbf{x}) = \begin{cases} 1 & \text{if } \sum_{i=0}^{d+1} w_i x_i > 0, \\ 0. & \text{otherwise.} \end{cases}$$

We want to find a weight vector $\mathbf{w}^*$ to minimize the number of classification errors produced. Namely, we want to minimize the error function

$$E(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^N (h(\mathbf{x}_i) - y_i)^2;$$

The optimization problem is

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} E(\mathbf{w}).$$

Notice that solving this optimization problem is difficult because $h(\cdot)$ is non-differentiable

$$\lim_{\alpha \rightarrow 0^-} h(\alpha) = 0 \neq \lim_{\alpha \rightarrow 0^+} h(\alpha) = 1.$$

The optimization problem can be solved for *separable data* using the (one layer) *perceptron algorithm*. (Please refer to the Python notebook with the study materials.)

We can *relax* the binary classification and use a differentiable *activation* function instead. The sigmoid function is a good candidate (Why?) The new hypothesis space is as follows.

$$\mathcal{H} = \{h(\mathbf{x}) : h(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x}), \mathbf{w} \in \mathbb{R}^{d+1}\},$$

where

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{1 + e^x}.$$

fig/perceptronplane.png

Figure 2: Separating hyperplane.

Notice that

$$\frac{d}{dx}\sigma(x) = \sigma(x)(1 - \sigma(x)).$$

Again, we use a the error function

$$E(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^N (h(\mathbf{x}_i) - y_i)^2;$$

$E(\mathbf{w})$ can be minimized w.r.t. $\mathbf{w}$ using a *gradient descent* optimization algorithm, for which we need the gradient of $E(\mathbf{w})$. Namely,

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} E(\mathbf{w}).$$

We solve this problem by computing the gradient and equating the gradient to zero.

$$\nabla E(\mathbf{w}^*) = \mathbf{0}.$$

The gradient is:

$$\nabla E(\mathbf{w}) = \sum_{i=1}^N (\sigma(\mathbf{w}^T \mathbf{x}_i) - y_i) \sigma(\mathbf{w}^T \mathbf{x}_i) (1 - \sigma(\mathbf{w}^T \mathbf{x}_i)) \mathbf{x}_i$$

Notice that the j -th component of the gradient is:

$$\begin{aligned} \frac{\partial}{\partial w_j} E(\mathbf{w}) &= \frac{\partial}{\partial w_j} \frac{1}{2} \sum_{i=1}^N (h(\mathbf{x}_i) - y_i)^2 \\ &= \sum_{i=1}^N (\sigma(\mathbf{w}^T \mathbf{x}_i) - y_i) \frac{\partial}{\partial w_j} (\sigma(\mathbf{w}^T \mathbf{x}_i) - y_i) \\ &= \sum_{i=1}^N (\sigma(\mathbf{w}^T \mathbf{x}_i) - y_i) \frac{\partial}{\partial w_j} \sigma(\mathbf{w}^T \mathbf{x}_i) \\ &= \sum_{i=1}^N (\sigma(\mathbf{w}^T \mathbf{x}_i) - y_i) \sigma(\mathbf{w}^T \mathbf{x}_i) (1 - \sigma(\mathbf{w}^T \mathbf{x}_i)) \frac{\partial}{\partial w_j} \mathbf{w}^T \mathbf{x}_i \\ &= \sum_{i=1}^N (\sigma(\mathbf{w}^T \mathbf{x}_i) - y_i) \sigma(\mathbf{w}^T \mathbf{x}_i) (1 - \sigma(\mathbf{w}^T \mathbf{x}_i)) \frac{\partial}{\partial w_j} \sum_{k=1}^d w_k x_{kj} \\ &= \sum_{i=1}^N (\sigma(\mathbf{w}^T \mathbf{x}_i) - y_i) \sigma(\mathbf{w}^T \mathbf{x}_i) (1 - \sigma(\mathbf{w}^T \mathbf{x}_i)) \sum_{k=1}^d \frac{\partial}{\partial w_j} w_k x_{kj} \\ &= \sum_{i=1}^N (\sigma(\mathbf{w}^T \mathbf{x}_i) - y_i) \sigma(\mathbf{w}^T \mathbf{x}_i) (1 - \sigma(\mathbf{w}^T \mathbf{x}_i)) \sum_{k=1}^d \delta_{kj} x_{kj} \\ &= \sum_{i=1}^N (\sigma(\mathbf{w}^T \mathbf{x}_i) - y_i) \sigma(\mathbf{w}^T \mathbf{x}_i) (1 - \sigma(\mathbf{w}^T \mathbf{x}_i)) x_{ij} \\ &= \sum_{i=1}^N c_i x_{ij}. \end{aligned}$$

$$\nabla E(\mathbf{w}) = \sum_{i=1}^N c_i \mathbf{x}_i.$$

What happens with the value of the gradient when data are separable?

Gradient Descent

The gradient descent algorithm pseudocode is the following.

Require: $f(\mathbf{x})$: Function to minimize, ∇f : Gradient of f , $\mathbf{x}_0$: Initial guess, α : Learning rate, ϵ : Tolerance

```

1:  $\mathbf{x} \leftarrow \mathbf{x}_0$ 
2: repeat
3:    $\mathbf{g} \leftarrow \nabla f(\mathbf{x})$                                 {Compute gradient}
4:    $\mathbf{x} \leftarrow \mathbf{x} - \alpha \mathbf{g}$                         {Update position}
5: until  $\|\mathbf{g}\| < \epsilon$                                     {Convergence criterion}
6: return  $\mathbf{x}$                                               {Return optimized value}
```

Algorithm 1: Gradient Descent

Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is an optimization algorithm commonly used in machine learning to minimize a loss function by iteratively adjusting the model's parameters. It is a variation of the gradient descent algorithm that updates parameters using a single data point (or a small batch) rather than the entire dataset, which makes it computationally efficient for large datasets.

Steps of Stochastic Gradient Descent:

1. Initialize Parameters: Start with random initial values for the model's parameters.
2. Iterate Over Data:
 - (a) Randomly shuffle the dataset.
 - (b) For each data point (or small batch), compute the gradient of the *loss function* with respect to the parameters.
 - (c) Update the parameters using the computed gradient and a predefined learning rate.
3. Repeat: Continue until a stopping criterion is met (e.g., a maximum number of iterations or a sufficiently small loss).

SGD is widely used in training machine learning models, especially neural networks, due to its simplicity and scalability. However, it can suffer from high variance in updates, which may lead to slower convergence. Techniques like momentum, learning rate decay, and mini-batch SGD are often used to improve its performance.

Algorithm 2: Stochastic Gradient Descent (SGD)

Require: Dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, learning rate η , loss function $\mathcal{L}(\theta; x, y)$

Ensure: Optimized parameters θ

- 1: Initialize parameters θ randomly
 - 2: **while** stopping criterion not met **do**
 - 3: Shuffle dataset $\mathcal{D}$
 - 4: **for** each data point $(x_i, y_i) \in \mathcal{D}$ **do**
 - 5: Compute gradient $g = \nabla_{\theta} \mathcal{L}(\theta; x_i, y_i)$
 - 6: Update parameters $\theta \leftarrow \theta - \eta g$
 - 7: **end for**
 - 8: **end while**
 - 9: **return** θ
-

Mini-batch Stochastic Gradient Descent

Mini-batch Stochastic Gradient Descent (Mini-batch SGD) is an optimization algorithm commonly used in machine learning to find the optimal parameters of a model. It is a compromise between Stochastic Gradient Descent (SGD), which uses a single data point at a time, and Batch Gradient Descent, which uses the entire dataset. Instead, Mini-batch SGD processes a small subset of the data (mini-batch) in each iteration.

Steps of Mini-batch SGD:

- Initialize Parameters: Randomly initialize the model's parameters.
- Split Dataset into Mini-batches: Divide the training dataset into smaller, equally sized subsets (mini-batches).
- Iterate Over Mini-batches:
- For each mini-batch, compute the gradient of the loss function with respect to the model parameters.
- Update the model parameters using the computed gradient and a predefined learning rate.
- Repeat: Continue until a stopping criterion is met (e.g., convergence or maximum iterations).

This algorithm has several advantages: combines the efficiency of SGD with the stability of batch gradient descent, makes use of vectorized operations, leading to faster computation, reduces noise compared to SGD.

Require: Dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, learning rate η , loss function $\mathcal{L}(\theta; x, y)$, mini-batch size m

Ensure: Optimized parameters θ

```

1: Initialize parameters  $\theta$  randomly
2: while stopping criterion not met do
3:   Shuffle dataset  $\mathcal{D}$ 
4:   Split  $\mathcal{D}$  into mini-batches  $\{\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_k\}$  where  $|\mathcal{B}_j| = m$ 
5:   for each mini-batch  $\mathcal{B}_j$  do
6:     Compute gradient  $g = \frac{1}{m} \sum_{(x,y) \in \mathcal{B}_j} \nabla_{\theta} \mathcal{L}(\theta; x, y)$ 
7:     Update parameters  $\theta \leftarrow \theta - \eta g$ 
8:   end for
9: end while
10: return  $\theta$ 

```

Algorithm 3: Mini-batch Stochastic Gradient Descent (Mini-batch SGD)

Box 3: Classifying NIST digits

Download the NIST digits data set from Yann Lecun website (<https://yann.lecun.com/exdb/mnist/>). Choose two out of the ten classes and implement a linear classifier to predict the class label of a new digit.

Appendix

Box 4: Application Example (Pairs of dice)

You are told that there are two pairs of coins. The first pair is fair,

$$\theta_1(H) = \theta_1(T) = \frac{1}{2}.$$

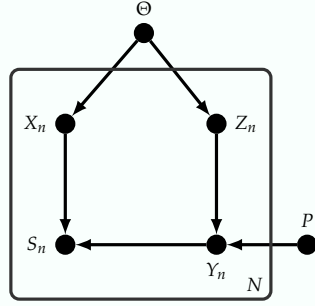
The second pair is biased as both coins have probability

$$\theta_2(H) = \frac{2}{3}, \theta_2(T) = \frac{1}{3}$$

of producing heads and tails, respectively. One of the two pairs is chosen at random with probability 0.5 and thrown 10 times. The sum of the coins is recorded for each throw.

1. If the throw results in the sequence "1010101010"? Which pair of dice was picked for the throw?
2. Repeat (1) for the sequence "2222200000".
3. Assume that the result of the second coin is flipped (F) with probability $p = 0.5$ before recording the result of the throw with probability. Repeat 1 and 2 under this assumption. Consider the case for which $p = 0.2$. Repeat exercises 1 and 2.

The graphical model associated with this problem is as follows.



- X_n represents the n -th result from the throw of the first coin; $X_n(H) = 1$ and $X_n(T) = 0$, and

$$p(x_n | \theta) = \theta^{x_n} (1 - \theta)^{(1-x_n)}.$$

- Z_n represents the unobserved result from the n -th throw of the second coin; $Z_n(F) = 1$ and $Z_n(F^c) = 0$, and

$$p(z_n | \theta) = \theta^{z_n} (1 - \theta)^{(1-z_n)}.$$

- Y_n represents the recorded result from the n -th throw of the second coin; $Y_n(H) = 1$ and $Y_n(T) = 0$, and

$$p(y_n | z_n, p) = p^{I(y_n \neq z_n)} (1 - p)^{I(y_n = z_n)}.$$

- S_n is a deterministic function of X_n and Y_n ,

$$S_n = X_n + Y_n,$$

$$p(s_n | x_n, y_n) = I(s_n = x_n + y_n),$$

and

$$I(e) = \begin{cases} 1, & \text{if } e = \text{true;} \\ 0, & \text{otherwise.} \end{cases}$$

Let $\mathbf{X} = (X_1, X_2, \dots, X_N)$, $\mathbf{Y} = (Y_1, Y_2, \dots, Y_N)$, $\mathbf{Z} = (Z_1, Z_2, \dots, Z_N)$, $\mathbf{S} = (S_1, S_2, \dots, S_N)$, $f_n = I(y_n \neq z_n) = 1 - I(y_n = z_n)$, and $\mathbf{f} = (f_1, f_2, \dots, f_N)$. (f_n indicates whether the second was flipped or not.) The joint probability distribution for $\mathbf{x}, \mathbf{z}, \mathbf{y}, \mathbf{s}$ given θ and p is

$$\begin{aligned} p(\mathbf{x}, \mathbf{z}, \mathbf{y}, \mathbf{s} \mid \theta, p) &= \prod_{n=1}^N p(x_n \mid \theta) p(z_n \mid \theta) p(y_n \mid z_n, p) p(s_n \mid x_n, y_n) \\ &= \prod_{n=1}^N \theta^{x_n} (1 - \theta)^{(1-x_n)} \theta^{z_n} (1 - \theta)^{(1-z_n)} p^{I(y_n \neq z_n)} (1 - p)^{I(y_n = z_n)} I(s_n = x_n + y_n). \\ &= \theta^{\sum_{n=1}^N x_n} (1 - \theta)^{\sum_{n=1}^N (1-x_n)} \theta^{\sum_{n=1}^N z_n} (1 - \theta)^{\sum_{n=1}^N (1-z_n)} p^{\sum_{n=1}^N I(y_n \neq z_n)} (1 - p)^{\sum_{n=1}^N I(y_n = z_n)} \end{aligned}$$

Notice that

$$\begin{aligned} x_n &= s_n - y_n, \\ y_n &= z_n(1 - f_n) + (1 - z_n)f_n, \end{aligned}$$

and

$$x_n = s_n - z_n + 2z_n f_n - f_n.$$

We define N_1^x , and N_1^z as follows,

$$N_x = \sum_{n=1}^N (s_n - z_n + 2z_n f_n - f_n) = N_s - N_z + 2N_{zf} - N_f \geq 0,$$

$$N_s = \sum_{n=1}^N s_n,$$

$$N_z = \sum_{n=1}^N z_n,$$

and

$$N_{zf} = \sum_{n=1}^N z_n f_n,$$

$$N_f = \sum_{n=1}^N f_n.$$

Therefore we can write the probability distribution function as follows,

$$\begin{aligned} p(\mathbf{x}, \mathbf{z}, \mathbf{y}, \mathbf{s} \mid \theta, p) &= \theta^{\sum_{n=1}^N x_n} (1 - \theta)^{\sum_{n=1}^N (1-x_n)} \theta^{\sum_{n=1}^N z_n} (1 - \theta)^{\sum_{n=1}^N (1-z_n)} p^{\sum_{n=1}^N I(y_n \neq z_n)} (1 - p)^{\sum_{n=1}^N I(y_n = z_n)} \\ &= \theta^{N_x} (1 - \theta)^{(N - N_x)} \theta^{N_z} (1 - \theta)^{(N - N_z)} p^{N_f} (1 - p)^{N - N_f} \\ &= \theta^{N_x + N_z} (1 - \theta)^{(2N - (N_x + N_z))} p^{N_f} (1 - p)^{N - N_f} \\ &= p(\mathbf{s}, \mathbf{z}, \mathbf{f} \mid \theta, p) \end{aligned}$$

Observe that $N_x = N_x(\mathbf{s}, \mathbf{z}, \mathbf{f})$, $N_z = N_z(\mathbf{z})$, and therefore

$$p(\mathbf{x}, \mathbf{z}, \mathbf{y}, \mathbf{s} \mid \theta, p) = p(\mathbf{s}, \mathbf{z}, \mathbf{f} \mid \theta, p),$$

and

$$p(\mathbf{s} \mid \theta, p) = \sum_{\mathbf{x}} \sum_{\mathbf{z}} \sum_{\mathbf{y}} p(\mathbf{x}, \mathbf{z}, \mathbf{y}, \mathbf{s} \mid \theta, p) = \sum_{\mathbf{f}} \sum_{\mathbf{z}} p(\mathbf{s}, \mathbf{z}, \mathbf{f} \mid \theta, p).$$

Box 5: Decision Rule

$$\mathbb{I} \left(\frac{p(\theta_1 \mid \mathbf{s}, p)}{p(\theta_1 \mid \mathbf{s}, p)} > 1 \right) \Rightarrow \text{choose } \theta_1,$$

which is equivalent to

$$\mathbb{I} \left(\frac{p(\mathbf{s} \mid \theta_1, p)p(\theta_1)}{p(\mathbf{s} \mid \theta_2, p)p(\theta_2)} > 1 \right) \Rightarrow \text{choose } \theta_1,$$

since

$$p(\theta \mid \mathbf{s}, p) \propto p(\mathbf{s} \mid \theta, p)p(\theta) = \sum_{\mathbf{f}} \sum_{\mathbf{z}} p(\mathbf{s}, \mathbf{z}, \mathbf{f} \mid \theta, p)p(\theta).$$

Notice that

$$p(\mathbf{s} \mid p) = \sum_{\theta} \sum_{\mathbf{f}} \sum_{\mathbf{z}} p(\mathbf{s}, \mathbf{z}, \mathbf{f} \mid \theta, p)p(\theta),$$

with $\theta \in \{\theta_1, \theta_2\}$.

These equations lead to the following results regarding question 3.

$$\frac{p([2, 2, 2, 2, 2, 0, 0, 0, 0] \mid \frac{2}{3}, \frac{2}{3})p(\frac{1}{2})}{p([2, 2, 2, 2, 2, 0, 0, 0, 0] \mid \frac{1}{2}, \frac{1}{2})p(\frac{1}{2})} = 1.80 \Rightarrow \theta_2 = \frac{2}{3}.$$

$$\frac{p([1, 0, 1, 0, 1, 0, 1, 0, 1, 0] \mid \frac{2}{3}, \frac{2}{3})p(\frac{1}{2})}{p([1, 0, 1, 0, 1, 0, 1, 0, 1, 0] \mid \frac{1}{2}, \frac{1}{2})p(\frac{1}{2})} = 0.13 \Rightarrow \theta_1 = \frac{1}{2}.$$

Also, notice that

$$\mathbb{E}[\mathbf{s} \mid \theta, p] = \sum_{\mathbf{f}} \sum_{\mathbf{z}} \mathbf{s} p(\mathbf{s}, \mathbf{z}, \mathbf{f} \mid \theta, p),$$

$$\mathbb{E}[N_s \mid \theta, p] = \sum_{\mathbf{f}} \sum_{\mathbf{z}} N_s p(\mathbf{s}, \mathbf{z}, \mathbf{f} \mid \theta, p).$$

The effect of θ on N_s is

$$e(\theta_1, \theta_2 \mid p) = \mathbb{E}[N_s \mid \theta_1, p] - \mathbb{E}[N_s \mid \theta_2, p]$$

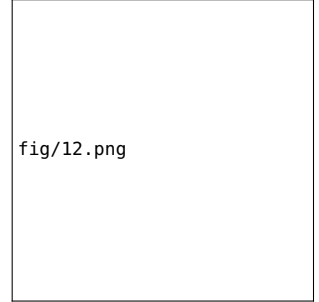


Figure 3: Probability distribution $p(N_s)$ in Box for $\theta = \frac{1}{2}$ for problem 3 in Box 9. The expected value $\mu_{N_s} = 5$ and the entropy is $E(N_s) = 2.71$.

fig/1-99.100.png	fig/1-8.9.png	fig/1-4.5.png
fig/4.5.png	fig/8.9.png	fig/99.100.png

Figure 4: From left to right and from top to bottom, probability distributions $p(N_s)$ in Box for $\theta = \frac{1}{100}, \frac{1}{9}, \frac{1}{5}, \frac{99}{100}, \frac{8}{9}, \frac{4}{5}$ for problem 3 in Box 9. The corresponding mean values, μ_{N_s} , are 2.55, 3.06, 3.5, 6.5, 6.9, and 7.45 respectively. The corresponding entropies, $E(N_s)$, are 2.44, 2.56, 2.56, 2.44, and 2.22, respectively.